

A Borrow-Aware Compiler from MIRLite to OSEA-IR

ANONYMOUS AUTHOR(S)

MIRLite gives typed Rust-like places an operational semantics on byte-addressed memory: every integer has its width, every aggregate its byte layout, and every pointer byte carries its provenance. Its compiler lowers each statement to OSEA-IR with explicit reads, retags, writes, and borrow retirement. We present Stacked Borrows, both languages, and the compiler in one format: a grammar, a table of named rules, and the stepwise execution of one five-statement program that allocates a nested tuple, writes a nested field, borrows that field, and writes through the borrow. We then state the forward simulation that relates source and target memory byte by byte, locals, and per-byte permission stacks, and replay the same program as a simulation. The simulation is mechanized in Lean 4 for every MIRLite program, under two conditions on the byte layouts that a decidable check discharges for the layouts Rust gives.

Additional Key Words and Phrases: borrow semantics, compiler correctness, Lean, Stacked Borrows

1 Introduction

The compiler in this paper connects two views of the same memory operation. MIRLite says *which typed place* a Rust-like statement accesses. OSEA-IR says *which reads, retags, writes, and borrow retirements* implement that access. The proof relates their executions without demanding that their permission stacks be literally equal: the target is allowed to introduce short-lived tags that have no source counterpart. Section 2.4 names them route tags.

Memory is byte-addressed, as in Rust’s own interpreter. Every local has a *byte layout*: an integer occupies its width in bytes, a pointer eight, and an aggregate places its fields at byte offsets inside a block that may contain padding. A memory μ maps each address to an *abstract byte*, uninitialized or a concrete byte that may carry pointer provenance. A value is read and written *leaf by leaf*: the leaves of a layout are its integers and pointers, each decoded from, or encoded into, its own bytes. A pointer value $\text{ptr}(b, o, e, sz, t)$ denotes address $b + o$ in an allocation with base b and size sz bytes, claims e bytes from there, and carries the provenance tag t . A permission state Π records one borrow stack per byte.

Section 2 formalizes the compilation. It (1) fixes the permission model, per-byte Stacked Borrows, and threads it through every memory operation (Section 2.1); (2) defines MIRLite, the typed source language (Section 2.2); (3) defines OSEA-IR, a register machine whose instructions expose the permission events a MIRLite statement leaves implicit (Section 2.3); (4) presents the compiler between them (Section 2.4); and (5) states the forward simulation that the compiler satisfies (Section 2.5). Section 3 records what is mechanized, and Appendix A extends every table from the constructs shown in the main text to the full executable surface.

READING GUIDE Subsections 2.1 to 2.4 share one layout. *Syntax* is a single figure: grammar, semantic domains, types. *Semantics* is a single table of named rules, read as premises, conclusion, name. The *example* is a stepwise table giving the state after each instruction, and a walkthrough that names the rule each row fires. From Section 2.2 on the example is one program, read once more in Section 2.5 as a simulation.

2 Compiling MIRLite to OSEA-IR with ownership

This section formalizes the compilation from MIRLite to OSEA-IR and the sense in which it preserves ownership. The order is bottom-up. Stacked Borrows comes first (Section 2.1), because both languages are defined over it. MIRLite (Section 2.2) and OSEA-IR (Section 2.3) follow, then the

43 $\Pi(a) = [] \xrightarrow{\text{own}(a)} [\text{Own}(t)] \xrightarrow{\text{ref}(\text{mutable}, a, t)} [\text{MutRef}(u), \text{Own}(t)] \xrightarrow{\text{useMut}(a, u)} [\text{MutRef}(u), \text{Own}(t)] \xrightarrow{\text{die}(a, u)} [\text{Own}(t)]$

44 Fig. 1. A Stacked Borrows command sequence on the one byte at address a , with the stack after each
 45 command.

46 compiler between them (Section 2.4), then its correctness (Section 2.5). Each subsection ends with
 47 an example. The example of Section 2.1 is a command sequence on a single byte; from Section 2.2
 48 on it is one MIRLite program, introduced there and carried through the rest of the section.

49 2.1 Stacked Borrows

50 The Stacked Borrows model of Jung et al. formalizes Rust’s ownership and borrowing rules for
 51 safe and unsafe code. It is orthogonal to the rest of a memory model: it is added to a language
 52 by threading a permission state through the memory operations. We use that to give MIRLite
 53 and OSEA-IR the *same* permission semantics. Formally, both languages are parameterized by a
 54 *permission model*, an abstract state Π with partial operations `own`, `read`, `useMut`, `ref`, and `die`.
 55 Each returns a new state or fails; a failure is undefined behavior and aborts the execution. The
 56 correctness result instantiates the model with the per-byte Stacked Borrows of this subsection,
 57 as Miri does. The full executable language uses four further operations, for deallocation, exposed
 58 provenance, and protector frames; they are in Appendix A.2.

59 A permission state is $\Pi = (\text{stacks}, \text{NextTag}, \text{frames}, \text{exposed})$. The component `stacks` is a partial
 60 map from byte addresses to *borrow stacks*; `NextTag` is the next fresh tag; `frames` and `exposed` are
 61 the protector frames and the list of exposed tags, which only the full language uses (Appendix A.2).
 62 We write $\Pi(a)$ for the stack at a and leave the other three components implicit when a rule does
 63 not change them. A borrow stack is a list of *items*, topmost first,

64 $\iota ::= \text{Own}(t) \mid \text{MutRef}(t) \mid \text{Ref}(t) \mid \text{RawPtr}(m', t) \mid \text{Disabled}(t),$

65 where t is the item’s tag and m' a mutability flag. The top of the stack is the most recently
 66 derived permission. A new item is derived from an existing one by a *retag*, and the *retag kind*

67 $k ::= \text{shared} \mid \text{mutable} \mid \text{raw-const} \mid \text{raw-mut} \mid \text{two-phase}$

68 says which item it pushes: a shared reference `Ref`, a mutable reference `MutRef`, a read-only or
 69 mutable raw pointer `RawPtr(false, ·)` or `RawPtr(true, ·)`, or a reserved mutable borrow. The kinds
 70 `shared` and `mutable` are Rust’s `&T` and `&mut T`, and `raw-const` and `raw-mut` are its raw pointers
 71 `*const T` and `*mut T`. A `raw-const` retag acts on the stack exactly as a `shared` one does, but
 72 pushes a differently named item; the name matters because related stacks must agree item by item
 73 (Definition 2.5). A retag also takes a protector flag c and an interior-mutability mask m , a list of
 74 booleans with one entry per byte of the retagged range. Both are inert in the main text, where $c =$
 75 `false` and m marks no byte, and so is `two-phase`; all three are defined in Appendix A.2. Table 1
 76 gives the rules. A rule $\langle \text{op}, \Pi \rangle \Downarrow_a \Pi'$ acts on one byte a ; “ u fresh” means $u = \Pi.\text{NextTag}$, and the
 77 conclusion increments the counter. Fig. 1 runs four of them: **SB-OWN** pushes the owning item
 78 of a new allocation, **SB-REF-MUT** derives a mutable borrow with a fresh tag u from its parent t ,
 79 **SB-USE-MUT** validates a write through u , and **SB-DIE** pops u ’s item, which is how a borrow is retired.
 80 After the four commands the stack is what it was after the first; only the tag counter remembers
 81 that u existed. That observation is what makes a route tag unobservable at the next statement
 82 boundary, and it is why the simulation of Section 2.5 compares tag counters by an inequality.

83 Three points of the model matter later. A read through t does not remove the mutable borrows
 84 above t : **SB-READ** *disables* them in place, because removing them would merge the raw-pointer
 85 groups on either side. A shared or raw-constant retag is a read through its parent followed by a
 86 push (**SB-REF-SHARED**). A raw-mutable retag performs no access and inserts its item directly above

Table 1. Stacked Borrows rules at one byte a . x and y range over lists of items, x being the part of the stack above the granting item; $\text{tag}(t)$ is the tag an item carries; $\text{dis}(x)$ replaces each $\text{MutRef}(u)$ in x by $\text{Disabled}(u)$. In **SB-REF-SHARED**, t' is the item that a retag of kind k pushes. No rule applies when a removed or disabled item is protected (Appendix A.2).

Premises	Conclusion	Rule Name
$\Pi(a) \in \{\perp, []\}, t \text{ fresh}$	$\langle \text{own}(a), \Pi \rangle \Downarrow_a (\Pi[a \mapsto [\text{Own}(t)]], t)$	SB-OWN
$\Pi(a) = x \# (\iota :: y), \text{tag}(\iota) = t,$ $\iota \neq \text{Disabled}(t)$	$\langle \text{read}(a, t), \Pi \rangle \Downarrow_a \Pi[a \mapsto \text{dis}(x) \# (\iota :: y)]$	SB-READ
$\Pi(a) = x \# (\iota :: y), \text{tag}(\iota) = t,$ $\iota \in \{\text{Own}(t), \text{MutRef}(t)\}$	$\langle \text{useMut}(a, t), \Pi \rangle \Downarrow_a \Pi[a \mapsto \iota :: y]$	SB-USE-MUT
$\langle \text{useMut}(a, t), \Pi \rangle \Downarrow_a \Pi_1, u \text{ fresh}$	$\langle \text{ref}(\text{mutable}, a, t), \Pi \rangle \Downarrow_a$ $(\Pi_1[a \mapsto \text{MutRef}(u) :: \Pi_1(a)], u)$	SB-REF-MUT
$\langle \text{read}(a, t), \Pi \rangle \Downarrow_a \Pi_1, u \text{ fresh},$ $(k, t') \in \{\text{shared}, \text{Ref}(u)\},$ $(\text{raw-const}, \text{RawPtr}(\text{false}, u))\}$	$\langle \text{ref}(k, a, t), \Pi \rangle \Downarrow_a$ $(\Pi_1[a \mapsto t' :: \Pi_1(a)], u)$	SB-REF-SHARED
$\Pi(a) = x \# (\iota :: y), \text{tag}(\iota) = t,$ $\iota \neq \text{Disabled}(t), u \text{ fresh}$	$\langle \text{ref}(\text{raw-mut}, a, t), \Pi \rangle \Downarrow_a$ $(\Pi[a \mapsto x \# (\text{RawPtr}(\text{true}, u) :: \iota :: y)], u)$	SB-REF-RAW
$\Pi(a) = \iota :: y, \text{tag}(\iota) = u,$ $\iota \neq \text{Own}(u)$	$\langle \text{die}(a, u), \Pi \rangle \Downarrow_a \Pi[a \mapsto y]$	SB-DIE
$\langle \text{op}(a + j, \dots), \Pi_j \rangle \Downarrow_a \Pi_{j+1} \text{ for } 0 \leq j < n,$ one fresh tag shared by all n bytes	$\text{op}(\Pi_0, a, n, \dots) = \Pi_n$	SB-RANGE

its parent (**SB-REF-RAW**), so sibling raw pointers share one group instead of invalidating each other. And every operation of the interface ranges over n bytes from a (**SB-RANGE**): $\text{own}(\Pi, a, n) = (\Pi', t)$, $\text{read}(\Pi, a, n, t) = \Pi'$, $\text{useMut}(\Pi, a, n, t) = \Pi'$, $\text{ref}(\Pi, a, n, t, k, c, m) = (\Pi', u)$, and $\text{die}(\Pi, a, n, t) = \Pi'$. The per-byte granularity is what makes the width of a borrow observable, down to a one-byte field inside a wider one, and hence what the compiler must get right in Section 2.4.

In the mechanization the operations are `sb_own`, `sb_read`, `sb_write`, `sb_ref`, and `sb_die` in `src/obseq3/sb.lean`; the paper writes `useMut` for `sb_write` to keep the source-level reading. The layer acts on address ranges and never asks what an address unit is.

2.2 MIRLite

MIRLite is a typed, sequential language for the memory-level part of Rust that matters to borrow reasoning (Fig. 2). A program is a list of statements. A statement assigns an expression to a place, or halts. A *place* is a local ℓ , a projection $p.q$ of a place along a path, or a dereference $*p$. Both p and d range over places; we write d when the place is the *destination* of an assignment, as in $d := e$, and p otherwise. An *expression* is a constant, a copy or a move of a place, or a reference to a place. A move is MIR’s move operand: like Miri, the semantics reads the source through a fresh mutable borrow that it retires at once (**E-MOVE**), so a move invalidates the shared borrows of the moved place where a copy only reads it. We do not model control flow in the main text; the guarded assignment of the full language is in Appendix A.

Types and layouts. A *layout type* τ is the static type of a place: an integer `Int` θ of integer type θ , a width in bits and a signedness; a pointer `Ptr` τ to a τ ; or a tuple. A *byte layout* β says where the bytes of a value of that type lie: an integer of n bytes, a thin pointer of eight bytes to a β , or a

TERMS

Program $\ni$ Stmt* halt
 Stmt $\ni$ s $\ni$ d := e | halt
 Place $\ni$ p $\ni$ ℓ | p.q | *p
 Place $\ni$ d
 Local $\ni$ ℓ = N
 Path $\ni$ q $\ni$ ε | f.q
 N $\ni$ f
 Expr $\ni$ e $\ni$ const(w) | copy(p) | move(p)
 | ref(k, c, m, p)
 Kind $\ni$ k $\ni$ shared | mutable
 | raw-const | raw-mut
 | two-phase
 Flag $\ni$ c = Bool
 Mask $\ni$ m = Bool*

SEMANTIC DOMAINS

State $\ni$ S = N $\times$ Env $\times$ Mem $\times$ Perm
 Env $\ni$ E = Local $\rightarrow$ N $\times$ Tag
 Mem $\ni$ μ = (N $\rightarrow$ Byte) $\times$ Allocs $\times$ N $\times$ N*
 Byte $\ni$ α $\ni$ uninit | init(x, π)
 Prov $\ni$ π $\ni$ $\perp$ | (b, sz, e, t)
 Value $\ni$ v $\ni$ undef | word(w)
 | ptr(b, o, e, sz, t)
 N $\ni$ i, a, b, o, e, sz, w
 Tag $\ni$ t, u = N
 Perm $\ni$ Π

TYPES AND BYTE LAYOUTS

Layout $\ni$ τ $\ni$ Int θ | Ptr τ
 | ($\tau_0, \dots, \tau_{n-1}$)
 Layout $\ni$ σ
 IntTy $\ni$ θ = N $\times$ Bool
 Ctx $\ni$ Γ = Layout*
 BLayout $\ni$ β $\ni$ int(n) | ptr(β)
 | tup($\bar{\beta}$, $\bar{o}$, n, al)
 Scalar $\ni$ κ $\ni$ int(n) | ptr
 LayEnv $\ni$ Λ = Local $\rightarrow$ BLayout

Fig. 2. Grammar, semantic domains, and types of MIRLite. Every line is headed $X \ni x$: the metavariable x ranges over the set X . With $\ni$ the line is a production: X is generated by the constructors on the right. With = the set X is built from other sets: $\times$ is a product, $\rightarrow$ a partial map, $\rightarrow$ a total one, and X^* the lists over X ; Perm is the set of permission states of Section 2.1. With nothing on the right it declares a further metavariable over a set defined elsewhere. Retag kinds k , flags c , and masks m are those of Section 2.1, repeated here. The main text covers these forms; Fig. 5 has the rest.

tuple whose fields $\bar{\beta}$ lie at the byte offsets $\bar{o}$ inside a block of n bytes aligned to al. Its size $|\beta|$ is n , eight, or n , and its *leaves* are the scalars it holds, with their offsets:

$$\text{leaves}(\text{int}(n)) = [(0, \text{int}(n))] \quad \text{leaves}(\text{ptr}(\beta)) = [(0, \text{ptr})] \quad \text{leaves}(\text{tup}(\bar{\beta}, \bar{o}, n, \text{al})) = \#_j [(o_j + o', \kappa) \mid (o', \kappa) \in \text{leaves}(\beta_j)].$$

A scalar has size $|\text{int}(n)| = n$ and $|\text{ptr}| = 8$. The layout type and the byte layout are two halves of one Rust type: the type says what a place holds, the layout where its bytes are. Keeping them apart lets the source take rustc's own field offsets for repr(Rust) structs, which reorder fields. A *layout environment* Λ gives every local a byte layout, and every place gets one statically: $\Lambda(\ell)$ for a local, the layout of the field that q selects for $p.q$, and β for $*p$ when $\Lambda(p) = \text{ptr}(\beta)$. The byte offset of a projection, $\text{off}_\Lambda(p, q)$, is the sum of the field offsets along q in $\Lambda(p)$. The theorem of Section 2.5 holds for every Λ whose integer and pointer places have one-leaf layouts of the right size (Definition 2.6).

A *local* ℓ is a program variable, the counterpart of a MIR local such as $_0$. A context Γ lists the layout types of a program's locals, a local is an index into that list, and $\ell : \tau$ means that entry ℓ of Γ is τ . We write x and y for the locals 0 and 1 of the running program. A *path* q is a list of tuple-field indices, read left to right: ε selects the whole region and $f.q$ selects field f and then follows q ; it is typed, $q : \sigma \rightarrow \tau$, when following it through a σ selects a τ . Places are indexed by the layout

type they select: $\Gamma \vdash \ell : \tau$ if $\ell : \tau \in \Gamma$; $\Gamma \vdash p.q : \tau$ if $\Gamma \vdash p : \sigma$ and $q : \sigma \rightarrow \tau$; and $\Gamma \vdash *p : \tau$ if $\Gamma \vdash p : \text{Ptr } \tau$. Expressions and statements are indexed the same way: $\text{const}(w)$ has any integer type, the one its destination has; $\text{copy}(p)$ and $\text{move}(p)$ have the type of p ; $\text{ref}(k, c, m, p)$ has $\text{Ptr } \tau$ when $\Gamma \vdash p : \tau$; and $d := e$ requires d and e to have the same type, so a `u8` is never copied into a `u64` without a cast.

Reference formation. $\text{ref}(k, c, m, p)$ carries the retag kind k , protector flag c , and mask m of Section 2.1 and hands them to the permission model (**E-REF**). The mask a program writes has one entry per leaf; $\text{bytes}_\beta(m)$ gives every byte of β the entry of the leaf it belongs to. The kind is not limited to Rust references: a MIRLite program can form raw pointers and two-phase borrows as well.

Semantic domains. A *value* v is the value of one leaf: undefined, a word w , or a pointer; $\bar{v}$ is a list of values, one per leaf. A pointer $\text{ptr}(b, o, e, sz, t)$ has five fields. The *base* b and *size* sz , in bytes, identify the allocation $[b, b + sz)$ it points into, and every access through the pointer is checked against those bounds. The *offset* o locates the pointer inside that allocation, so the address it denotes is $b + o$; pointer arithmetic changes o and nothing else. The *extent* e is the number of bytes the pointer claims from there, the referent's size for a thin pointer and the slice's length for a slice reference (Appendix A). The *tag* t is its provenance, the identity under which the permission model of Section 2.1 admits or rejects the access. Two pointers to the same address with different tags are different values, and only one of them may be allowed to write.

A memory μ is a total map from addresses to *bytes*, together with the table of allocated ranges, the bump-allocation watermark, and the list of freed bases. A byte is uninitialized or a concrete byte x that may carry a *provenance* π , the base, size, extent, and tag of the pointer it is part of. Values meet bytes at the leaves. Encoding $\text{enc}_\kappa(v)$ turns `undef` into $|\kappa|$ uninitialized bytes, a word into its $|\kappa|$ little-endian bytes without provenance (the word must fit), and $\text{ptr}(b, o, e, sz, t)$ into the little-endian bytes of the address $b + o$, every one carrying (b, sz, e, t) . Decoding dec_κ inverts it: an integer leaf is the little-endian value of its bytes, with any provenance stripped; a pointer leaf is a pointer when its eight bytes carry one common provenance; and any uninitialized byte makes the leaf `undef`. A typed read and write of a value of layout β at address a are

$$\text{rd}_\beta(\mu, a) = [\text{dec}_\kappa(\mu[a + o..a + o + |\kappa|]) \mid (o, \kappa) \in \text{leaves}(\beta)],$$

$$\text{wr}_\beta(\mu, a, \bar{v}) = \mu[a..a + |\beta|] \mapsto \text{uninit, then } a + o \mapsto \text{enc}_\kappa(v_j) \text{ for the } j\text{-th leaf } (o, \kappa),$$

so padding becomes uninitialized, as a typed copy does in MiniRust; the write fails when $\bar{v}$ has the wrong number of values or a word does not fit its leaf. The environment E maps a local either to no binding or to an allocation base and its owning tag. A state is $S = (i, E, \mu, \Pi)$ with i the program counter. Table 2 gives the rules; they use three judgments, which we introduce before the interpreter.

Place resolution ($\Downarrow_p$). The judgment $S \vdash p \Downarrow_p (\langle a, t, b, sz \rangle, \Pi')$ reads: in state S , the well-typed place p resolves to $\langle a, t, b, sz \rangle$, and the permission state becomes Π' . The four components of a *resolved place* are the address a the place denotes, the tag t through which it is to be accessed, and the base b and size sz of the allocation it lies in. It is to a place what a list of values is to an expression, the result of evaluating it, and it is never stored. The state is on the left because resolution consults all of it: the environment for a local, memory and permissions for a dereference. It does not write memory, but it can change the permission state. **PLC-LOCAL** reads the environment. **PLC-PROJ** is typed address arithmetic in bytes: it preserves provenance, bounds, and permissions. **PLC-DEREF** changes provenance, because the pointer *value*, not the place holding it, identifies the referent; loading that value is an eight-byte read through the tag of the place that holds it, and all eight bytes must lie inside the allocation. Nested dereferences thread these permission changes from

Table 2. MIRLite semantics. $S.i$, $S.E$, $S.\mu$, and $S.\Pi$ are the components of a state $S = (i, E, \mu, \Pi)$, and $S[\Pi \mapsto \Pi']$ is S with its permission state replaced by Π' . Γ and Δ are fixed throughout; P is the program being run. “ b live” means that b is not a freed base of $S.\mu$. All rules describe successful branches. An unbound local in a read, a freed or out-of-bounds access, a read of an uninitialized leaf, or a rejection by the permission model is an error and has no successor.

Premises	Conclusion	Rule Name
$S.E(\ell) = (a, t)$	$S \vdash \ell \Downarrow_p ((a, t, a, \Lambda(\ell)), S.\Pi)$	PLC-LOCAL
$S \vdash p \Downarrow_p ((a, t, b, sz), \Pi')$	$S \vdash p.q \Downarrow_p ((a + \text{off}_\Lambda(p, q), t, b, sz), \Pi')$	PLC-PROJ
$S \vdash p \Downarrow_p ((a, t, b, sz), \Pi_1)$, b live, $b \leq a$, $a + 8 \leq b + sz$, $\text{read}(\Pi_1, a, 8, t) = \Pi_2$, $\text{dec}_{\text{ptr}}(S.\mu[a..a + 8]) =$ $\text{ptr}(b', o', e', sz', t')$	$S \vdash *p \Downarrow_p ((b' + o', t', b', sz'), \Pi_2)$	PLC-DEREF
$\text{lookup}(S, d)$ is defined	$\text{prepare}(S, d) = S$	PREP-BOUND
$\text{lookup}(S, d)$ undefined, $\text{root}(d) = \ell$, $S.E(\ell) = \perp$, $\beta = \Lambda(\ell)$, $\text{alloc}(S.\mu, \beta , \text{al}(\beta)) = (b, \mu')$, $\text{own}(S.\Pi, b, \beta) = (\Pi', t)$	$\text{prepare}(S, d) = (S.i, S.E[\ell \mapsto (b, t)], \mu', \Pi')$	PREP-ALLOC
	$S \vdash \text{const}(w) \Downarrow_e ([\text{word}(w)], S)$	E-CONST
$S \vdash p \Downarrow_p ((a, t, b, sz), \Pi_1)$, $\beta = \Lambda(p)$, b live, $a + \beta \leq b + sz$, $\text{read}(\Pi_1, a, \beta , t) = \Pi_2$, $\bar{v} = \text{rd}_\beta(S.\mu, a)$, $\text{undef} \notin \bar{v}$	$S \vdash \text{copy}(p) \Downarrow_e (\bar{v}, S[\Pi \mapsto \Pi_2])$	E-COPY
$S \vdash p \Downarrow_p ((a, t, b, sz), \Pi_1)$, $\beta = \Lambda(p)$, b live, $a + \beta \leq b + sz$, $\text{ref}(\Pi_1, a, \beta , t, \text{mutable}, \text{false}, []) = (\Pi_2, u)$, $\text{read}(\Pi_2, a, \beta , u) = \Pi_3$, $\text{die}(\Pi_3, a, \beta , u) = \Pi_4$, $\bar{v} = \text{rd}_\beta(S.\mu, a)$, $\text{undef} \notin \bar{v}$	$S \vdash \text{move}(p) \Downarrow_e (\bar{v}, S[\Pi \mapsto \Pi_4])$	E-MOVE
$S \vdash p \Downarrow_p ((a, t, b, sz), \Pi_1)$, $\beta = \Lambda(p)$, $ \beta = 0$ or (b live and $a + \beta \leq b + sz$), $\text{ref}(\Pi_1, a, \beta , t, k, c, \text{bytes}_\beta(m)) = (\Pi_2, u)$	$S \vdash \text{ref}(k, c, m, p) \Downarrow_e ([\text{ptr}(b, a - b, \beta , sz, u)], S[\Pi \mapsto \Pi_2])$	E-REF
$P(S.i) = (d := e)$, $\beta = \Lambda(d)$, $\text{prepare}(S, d) = S_1$, $S_1 \vdash e \Downarrow_e (\bar{v}, S_2)$, $S_2 \vdash d \Downarrow_p ((a, t, b, sz), \Pi_3)$, b live, $a + \beta \leq b + sz$, $\text{useMut}(\Pi_3, a, \beta , t) = \Pi_4$, $\text{wr}_\beta(S_2.\mu, a, \bar{v}) = \mu'$	$S \xrightarrow{\text{mir}} (S.i + 1, S_2.E, \mu', \Pi_4)$	ASSGN
$P(S.i) = \text{halt}$ or $P(S.i) = \perp$	$S \xrightarrow{\text{mir}} S$	HALT

the inside out. A separate *pure lookup*, $\text{lookup}(S, p)$, follows the same address calculation but only decodes the pointer bytes, with neither a bounds check nor a read. It is used only to decide whether an assignment root must be allocated.

Expression evaluation ($\Downarrow_e$). The judgment $S \vdash e \Downarrow_e (\bar{v}, S')$ produces one value per leaf of the expression’s layout. **E-COPY** and **E-REF** resolve their place and then act on its entire byte range: one read, or one ref, of width $|\beta|$. A copy that would read an uninitialized leaf is undefined behavior, as in Miri. **E-MOVE** reads through a mutable borrow of the range that it retires at once. The pointer built by **E-REF** carries the fresh tag u , the allocation’s base and size, and the referent’s size as its extent, so later bounds checks are against the whole allocation. A reference to a zero-sized place touches no byte: as in Miri, it needs no live, in-bounds memory, so it may be formed through a dangling pointer.

Interpreter ($\rightarrow_{\text{mir}}$). A step fetches $P(i)$. **ASSGN** runs four phases in a fixed order: prepare the destination's root, evaluate the expression, resolve the destination, write. Preparation (**PREP-BOUND**, **PREP-ALLOC**) allocates the root local of d when it is unbound: the bump allocator returns a fresh base b aligned to the layout's alignment, own a fresh tag t over its $|\beta|$ bytes, and E is extended. A destination rooted through a dereference is never allocated implicitly; that case is an error. Crucially, *the whole expression is evaluated before the destination is resolved*. Hence $\text{copy}(p)$ materializes its values before a destination access can invalidate a tag that p needs. The write encodes the values at the *destination's* layout and validates a useMut over its whole range, padding included. The conclusion takes its environment from S_2 , the state after the expression, so allocation during preparation and any effect of the expression survive. We write $S \xRightarrow[n]{P} S'$ for n steps; halt and a missing statement are fixed points (**HALT**), so extra fuel is harmless.

Example. The running program of this paper declares a nested tuple x and a pointer y , and is small enough to execute by hand:

```
MIRLITE   $\Gamma = [x : (\text{u64}, (\text{u64}, \text{u64})), y : \text{Ptr u64}]$ 
0:  $x.0 := \text{const}(5)$ 
1:  $x.1.0 := \text{const}(42)$ 
2:  $y := \text{ref}(\text{mutable}, \text{false}, [], x.1.0)$ 
3:  $*y := \text{const}(7)$ 
4:  $\text{halt}$ 
```

Here u64 is $\text{Int}(64, \text{false})$, and Λ is the C layout: x is $\beta_x = \text{tup}([\text{int}(8), \text{tup}([\text{int}(8), \text{int}(8)], [0, 8], 16, 8)], [0, 8], 24, 8)$, 24 bytes with $x.1.0$ at byte offset $8 + 0 = 8$, and y is $\text{ptr}(\text{int}(8))$. Statement 0 allocates x , because an assignment to an unbound root allocates it. Statement 1 writes the eight bytes of $x.1.0$ in the middle of x ; it is the statement for which the compiler of Section 2.4 must introduce, and then retire, a tag the source never sees. Statement 2 allocates y and stores a mutable reference to the field just written. Statement 3 writes through that reference. Execution starts from the empty state: the bump allocator's watermark is 1, so that no allocation is ever at address 0, and tags are minted from 1 upward, tag 0 being reserved (Appendix A.2). The program thus touches 32 bytes: x occupies $[8, 32)$, its base aligned up to 8, and y occupies $[32, 40)$. All concrete addresses, tags, registers, and labels in this section are those of the mechanized semantics running this program.

Table 3. The MIRLite configuration after every statement of the running program. A column is one leaf, eight bytes: its decoded value over the borrow stack that each of its bytes carries (the eight are equal throughout this run). “—” is unallocated memory, shading marks what the statement changed, and “next” is $\Pi.\text{NextTag}$.

after statement	E	[8, 16] $x.0$	[16, 24] $x.1.0$	[24, 32] $x.1.1$	[32, 40] y	next
<i>initially</i>	—	—	—	—	—	1
0: $x.0 := \text{const}(5)$	$x \mapsto (8, 1)$	word(5) [Own(1)]	undef [Own(1)]	undef [Own(1)]	—	2
1: $x.1.0 := \text{const}(42)$	$x \mapsto (8, 1)$	word(5) [Own(1)]	word(42) [Own(1)]	undef [Own(1)]	—	2
2: $y := \text{ref}(\text{mutable}, \dots, x.1.0)$	$x \mapsto (8, 1)$ $y \mapsto (32, 2)$	word(5) [Own(1)]	word(42) [MutRef(3), Own(1)]	undef [Own(1)]	ptr(8, 8, 8, 24, 3) [Own(2)]	4
3: $*y := \text{const}(7)$	$x \mapsto (8, 1)$ $y \mapsto (32, 2)$	word(5) [Own(1)]	word(7) [MutRef(3), Own(1)]	undef [Own(1)]	ptr(8, 8, 8, 24, 3) [Own(2)]	4
4: halt	$x \mapsto (8, 1)$ $y \mapsto (32, 2)$	word(5) [Own(1)]	word(7) [MutRef(3), Own(1)]	undef [Own(1)]	ptr(8, 8, 8, 24, 3) [Own(2)]	4

Table 3 gives the whole configuration after each statement; the program counter, not shown, is $k + 1$ after statement k , and `halt` leaves it at 4. Read a row against the one above it: the shaded entries are exactly what the rules below change.

- 0 x is unbound, so **PREP-ALLOC** allocates the 24 bytes [8, 32], binds $x \mapsto (8, 1)$, and **SB-OWN** gives each byte the stack [Own(1)]. **E-CONST** produces [word(5)]. **PLC-LOCAL** and **PLC-PROJ** resolve $x.0$ to $\langle 8, 1, 8, 24 \rangle$. **ASSGN** validates a useMut of the eight bytes [8, 16] through tag 1, which **SB-USE-MUT** accepts without changing the stacks, and encodes 5 there; bytes [16, 32] stay uninitialized and decode to undef.
- 1 Preparation is **PREP-BOUND**: nothing is allocated. Two applications of **PLC-PROJ** resolve $(x.q_1).q_0$ to $\langle 16, 1, 8, 24 \rangle$: offset 8 for field 1, then 0 for its field 0. **ASSGN** writes the bytes [16, 24] through the owner’s tag. There is no source retag, so their stacks do not change, and the sibling $x.1.1$, bytes [24, 32], is untouched.
- 2 **PREP-ALLOC** allocates y at address 32 with owning tag 2, *before* the expression is evaluated. **E-REF** resolves $x.1.0$ as in statement 1, and **SB-REF-MUT** pushes MutRef(3) on its eight bytes only. The pointer $\text{ptr}(8, 8, 8, 24, 3)$, to address 16 with extent 8, is encoded into [32, 40]: the little-endian bytes of 16, each carrying the provenance (8, 24, 8, 3). The tag counter moves from 2 to 4: one tag for y , one for the reference.
- 3 **PLC-DEREF** resolves y to $\langle 32, 2, 32, 8 \rangle$, performs an eight-byte read of [32, 40] through tag 2 (**SB-READ**, which changes nothing here), decodes the pointer, and returns $\langle 16, 3, 8, 24 \rangle$, the address and tag *loaded from memory*. **ASSGN** writes 7 through tag 3, which is on top of the stacks of [16, 24], so they do not change either.
- 4 **HALT**: the configuration is a fixed point.

Formalization of the typed syntax, values, and source transition system: `src/obseq3/syntax.lean`, `src/obseq3/values.lean`, and `src/obseq3/mirlite.lean`; bytes and layouts: `src/obseq3/bytemem.lean` and `src/obseq3/bytelayout.lean`.

2.3 OSEA-IR

336 TERMS

337 Program $\ni Q = \mathbb{N} \rightarrow \text{Instr}$
 338 Instr $\ni I ::= r := h$
 339 | $\text{store}_\beta(r_s, r_p)$
 340 | $\text{storec}_\beta(\bar{v}, r_p)$
 341 | $\text{die}(r, n)$
 342 | halt
 343 Rhs $\ni h ::= \text{load}_\beta(r)$
 344 | alloc_β
 345 | $\text{borrow}(k, c, m, n, r, \delta)$
 346 Register $\ni r ::= R_0 \mid R_1 \mid \dots$
 347

SEMANTIC DOMAINS

State $\ni T = \mathbb{N} \times \text{RegFile} \times \text{Mem} \times \text{Perm}$
 RegFile $\ni R = \text{Register} \rightarrow \text{Value}^*$
 Mem $\ni \mu = (\mathbb{N} \rightarrow \text{Byte}) \times \text{Allocs} \times \mathbb{N} \times \mathbb{N}^*$
 Value $\ni v ::= \text{undef} \mid \text{dat}(w)$
 | $\text{ptr}(b, o, e, sz, t)$

COMPILER STATE

CState $\ni C = \mathbb{N} \times \mathbb{N} \times \text{Code} \times \text{LocalMap}$
 Code $\ni K = \mathbb{N} \rightarrow \text{Instr}$
 LocalMap $\ni L = \text{Local} \rightarrow \text{Register} \times \text{Layout}$
 Cleanup $\ni D = (\text{Register} \times \mathbb{N})^*$

348 Fig. 3. Grammar and semantic domains of OSEA-IR (left, top right), and the state of the compiler (bottom
 349 right). Bytes α , byte layouts β , and masks are those of Fig. 2. Fig. 5 has the remaining instructions.

350 OSEA-IR is a register machine that exposes the memory and permission events implicit in a
 351 MIRLite statement (Fig. 3). A source place is gone: an instruction names a register holding a
 352 concrete pointer, a byte offset, and a number of bytes. Consequently one source step may need
 353 several target steps and may create short-lived tags with no source counterpart. A program Q is a
 354 *partial* map from labels to instructions, which lets the compiler extend code without renumbering
 355 an earlier fragment. Instructions are register assignments $r := h$, stores of a register or of a constant
 356 through a pointer, the retirement die of a borrow, and halt . A right-hand side h loads through a
 357 pointer, allocates, or borrows. Every load, store, and allocation carries the byte layout of what it
 358 moves, so the target knows where each leaf lies exactly as the source does.

359 **Semantic domains.** Target values mirror source values one to one, with dat for a machine
 360 word; the pointer fields have the same meaning as at the source. A register holds a list of values,
 361 one per leaf of what was loaded into it; the register file is a finite shadowing map. Memory is
 362 the same byte memory as at the source, read and written with the same rd_β and wr_β , and the
 363 permission state is the same Stacked Borrows instance. A target state is $T = (j, R, \mu, \Pi)$. Table 4
 364 gives the rules.

365 **RHS evaluation** ($\Downarrow_h$). The judgment $T \vdash h \Downarrow_h (\bar{v}, T')$ evaluates a right-hand side to a list of
 366 values and a state. It may update memory or permissions but does not advance j . **RVAL-LOAD** checks
 367 liveness and the complete byte range, reads it through the pointer's tag, and decodes it leaf by
 368 leaf, failing on an uninitialized leaf exactly as **E-COPY** does. **RVAL-BORROW** retags n bytes at offset
 369 δ from the pointer and returns the same pointer, moved by δ , claiming n bytes, and carrying the
 370 fresh tag; like **E-REF** it checks liveness and bounds only when $n \neq 0$; offsets are natural numbers,
 371 so there is no negative-offset case. **RVAL-ALLOC** is the only rule of the main text that changes both
 372 memory and permissions.

373 **Interpreter** ($\rightarrow_{\text{sea}}$). **EXEC-ASSGN** evaluates its right-hand side *before* inserting the returned
 374 entry. The two stores share one write-through-pointer helper: extract one pointer from r_p , check
 375 liveness and the range, use Mut , encode, advance. They differ in where the values come from: a
 376 register for **EXEC-STORE**, the instruction for **EXEC-STOREC**. The encoding is the store's layout, so a
 377 store whose values do not fit its leaves fails. **EXEC-DIE** has no separate range check: its admissibility
 378 is exactly that of the permission operation. There is no global well-formedness premise on register
 379 files; the instruction that consumes a register checks the shape it needs, which keeps malformed

Table 4. OSEA-IR semantics. $T.R$, $T.\mu$, and $T.\Pi$ are components of a state $T = (j, R, \mu, \Pi)$, and $T[\dots \mapsto \dots]$ replaces components, as in Table 2; the instruction rules write the state out as a tuple instead. Q is the program being run. Every rule that reads a pointer from $R(r)$ requires the register to hold exactly one pointer; “ b live” is as in Table 2.

Premises	Conclusion	Rule Name
$T.R(r) = [\text{ptr}(b, o, e, sz, t)], \quad a = b + o, \quad b \text{ live},$ $a + \beta \leq b + sz, \quad \text{read}(T.\Pi, a, \beta , t) = \Pi',$ $\bar{v} = \text{rd}_\beta(T.\mu, a), \quad \text{undef} \notin \bar{v}$	$T \vdash \text{load}_\beta(r) \Downarrow_h (\bar{v}, T[\Pi \mapsto \Pi'])$	RVAL-LOAD
$\text{alloc}(T.\mu, \beta , \text{al}(\beta)) = (b, \mu'),$ $\text{own}(T.\Pi, b, \beta) = (\Pi', u)$	$T \vdash \text{alloc}_\beta \Downarrow_h$ $([\text{ptr}(b, 0, \beta , \beta , u)], T[\mu \mapsto \mu', \Pi \mapsto \Pi'])$	RVAL-ALLOC
$T.R(r) = [\text{ptr}(b, o, e, sz, t)], \quad a = b + o + \delta,$ $n = 0 \text{ or } (b \text{ live and } a + n \leq b + sz),$ $\text{ref}(T.\Pi, a, n, t, k, c, m) = (\Pi', u)$	$T \vdash \text{borrow}(k, c, m, n, r, \delta) \Downarrow_h$ $([\text{ptr}(b, o + \delta, n, sz, u)], T[\Pi \mapsto \Pi'])$	RVAL-BORROW
$Q(j) = (r := h),$ $(j, R, \mu, \Pi) \vdash h \Downarrow_h (\bar{v}, (j, R, \mu_1, \Pi_1))$	$(j, R, \mu, \Pi) \xrightarrow{\text{sea}}$ $(j + 1, R[r \mapsto \bar{v}], \mu_1, \Pi_1)$	EXEC-ASSGN
$Q(j) = \text{store}_\beta(r_s, r_p), \quad R(r_s) = \bar{v},$ $R(r_p) = [\text{ptr}(b, o, e, sz, t)], \quad a = b + o, \quad b \text{ live},$ $a + \beta \leq b + sz, \quad \text{useMut}(\Pi, a, \beta , t) = \Pi',$ $\text{wr}_\beta(\mu, a, \bar{v}) = \mu'$	$(j, R, \mu, \Pi) \xrightarrow{\text{sea}} (j + 1, R, \mu', \Pi')$	EXEC-STORE
$Q(j) = \text{storec}_\beta(\bar{v}, r_p),$ $R(r_p) = [\text{ptr}(b, o, e, sz, t)], \quad a = b + o, \quad b \text{ live},$ $a + \beta \leq b + sz, \quad \text{useMut}(\Pi, a, \beta , t) = \Pi',$ $\text{wr}_\beta(\mu, a, \bar{v}) = \mu'$	$(j, R, \mu, \Pi) \xrightarrow{\text{sea}} (j + 1, R, \mu', \Pi')$	EXEC-STOREC
$Q(j) = \text{die}(r, n), \quad R(r) = [\text{ptr}(b, o, e, sz, t)],$ $\text{die}(\Pi, b + o, n, t) = \Pi'$	$(j, R, \mu, \Pi) \xrightarrow{\text{sea}} (j + 1, R, \mu, \Pi')$	EXEC-DIE
$Q(j) = \text{halt or } Q(j) = \perp$	$T \xrightarrow{\text{sea}} T$	EXEC-HALT

programs executable with explicit errors. We write $T \xrightarrow[n]{Q} T'$ for exactly n steps. Iteration does not stop early at a fixed point (EXEC-HALT), which makes the target step count of the simulation existential without a separate reflexive-transitive closure.

Example. Table 5 executes the code the compiler emits for the running program (Fig. 4), one group of labels per source statement. Compare each group’s last row with the corresponding row of Table 3: the bytes agree, the stacks agree up to the tags, and the tag counter runs ahead.

Table 5. The OSEA-IR configuration after every instruction of the compiled running program, in the format of Table 3 (memory leaves decoded as there). The R column shows the binding the instruction adds; registers are never removed, so the register file is the union of the rows above. After label k the program counter is $k + 1$. β_x is x 's layout, i8 abbreviates $\text{int}(8)$; mut abbreviates mutable, false with a mask that marks no byte. Labels 0–1, 2–4, 5–7, 8–9, and 10 are source statements 0 to 4.

after instruction	R gains	[8, 16] $x.0$	[16, 24] $x.1.0$	[24, 32] $x.1.1$	[32, 40] y	next
<i>initially</i>	—	—	—	—	—	1
0: $R_0 := \text{alloc}_{\beta_x}$	$R_0 \mapsto \text{ptr}(8, 0, 24, 24, 1)$	undef [Own(1)]	undef [Own(1)]	undef [Own(1)]	—	2
1: $\text{storec}_{i8}([\text{dat}(5)], R_0)$	—	word(5) [Own(1)]	undef [Own(1)]	undef [Own(1)]	—	2
2: $R_1 := \text{borrow}(\text{mut}, 8, R_0, 8)$	$R_1 \mapsto \text{ptr}(8, 8, 8, 24, 2)$	word(5) [Own(1)]	undef [MutRef(2), Own(1)]	undef [Own(1)]	—	3
3: $\text{storec}_{i8}([\text{dat}(42)], R_1)$	—	word(5) [Own(1)]	word(42) [MutRef(2), Own(1)]	undef [Own(1)]	—	3
4: $\text{die}(R_1, 8)$	—	word(5) [Own(1)]	word(42) [Own(1)]	undef [Own(1)]	—	3
5: $R_2 := \text{alloc}_{\text{ptr}(i8)}$	$R_2 \mapsto \text{ptr}(32, 0, 8, 8, 3)$	word(5) [Own(1)]	word(42) [Own(1)]	undef [Own(1)]	undef [Own(3)]	4
6: $R_3 := \text{borrow}(\text{mut}, 8, R_0, 8)$	$R_3 \mapsto \text{ptr}(8, 8, 8, 24, 4)$	word(5) [Own(1)]	word(42) [MutRef(4), Own(1)]	undef [Own(1)]	undef [Own(3)]	5
7: $\text{store}_{\text{ptr}(i8)}(R_3, R_2)$	—	word(5) [Own(1)]	word(42) [MutRef(4), Own(1)]	undef [Own(1)]	$\text{ptr}(8, 8, 8, 24, 4)$ [Own(3)]	5
8: $R_4 := \text{load}_{\text{ptr}(i8)}(R_2)$	$R_4 \mapsto \text{ptr}(8, 8, 8, 24, 4)$	word(5) [Own(1)]	word(42) [MutRef(4), Own(1)]	undef [Own(1)]	$\text{ptr}(8, 8, 8, 24, 4)$ [Own(3)]	5
9: $\text{storec}_{i8}([\text{dat}(7)], R_4)$	—	word(5) [Own(1)]	word(7) [MutRef(4), Own(1)]	undef [Own(1)]	$\text{ptr}(8, 8, 8, 24, 4)$ [Own(3)]	5
10: halt	—	word(5) [Own(1)]	word(7) [MutRef(4), Own(1)]	undef [Own(1)]	$\text{ptr}(8, 8, 8, 24, 4)$ [Own(3)]	5

0–1 **RVAL-ALLOC** allocates [8, 32) and issues **SB-OWN**, so R_0 holds the owner's pointer with tag 1. **EXEC-STOREC** encodes 5 into [8, 16) through it. The state equals the source's after statement 0.

2–4 One source write takes three instructions, and their permission trace is Fig. 1 at every byte of [16, 24), with $t = 1$ and $u = 2$. **RVAL-BORROW** issues **SB-REF-MUT** on those eight bytes alone and mints tag 2; **EXEC-STOREC** writes through tag 2; **EXEC-DIE** issues **SB-DIE**, after which their stacks are again [Own(1)]. The memory and the stacks now equal the source's after statement 1, but the tag counter is 3 where the source's is 2.

5–7 **RVAL-ALLOC** allocates y , **RVAL-BORROW** mints the reference, and **EXEC-STORE** stores it. Because the counter ran ahead, y 's owning tag is 3 here and 2 at the source, and the stored reference carries tag 4 here and 3 at the source: the bytes of [32, 40) are the same address bytes on both sides, with provenance tags 4 and 3.

8–9 **RVAL-LOAD** reads [32, 40) through tag 3 and leaves $\text{ptr}(8, 8, 8, 24, 4)$ in R_4 ; **EXEC-STOREC** writes [16, 24) through the loaded tag 4.

10 EXEC-HALT. R_1 still holds a pointer with the dead tag 2; registers are not observable, so the simulation of Section 2.5 ignores it.

Formalization of target values, RHS evaluation, instruction stepping, and fuelled execution: `src/obseq3/oseair.lean` (registers and values: `src/obseq3/values.lean`).

2.4 Compiler

The compiler is a state-passing translation from typed MIRLite syntax to OSEA-IR code, parameterized by the layout environment Λ the source runs on. Its obligation is to preserve the source evaluation order while introducing explicit loads, retags, stores, and the retirement of the tags it introduces. A compiler state is $C = (n_r, n_l, K, L)$ (Fig. 3, bottom right): n_r and n_l are the next fresh register and label, K is the emitted code map, and L maps a source local to a register and its layout type. Write $C \oplus [I_0, \dots, I_{n-1}]$ for the state that installs the list at labels $[n_l, n_l + n]$ and advances n_l by n , and “ r fresh in C ” for $r = R_{n_r}$ with n_r advanced. Every compiler computation is monotone: counters do not decrease, code below the old n_l is unchanged, and every old entry of L remains. A cleanup list D records borrows to retire; $\text{cleanup}(D)$ reverses D and maps each entry (r, n) to $\text{die}(r, n)$. Table 6 gives the rules; every offset and length in them is in bytes and comes from Λ .

Establishing roots. Writing an unbound root must mirror source preparation. **ROOT-BOUND** emits nothing; **ROOT-ALLOC** allocates the root local of the destination at its byte layout and records its register. For a dereference the root is the pointer local. The source never allocates a pointee implicitly and fails instead, so a forward simulation never observes that case. Read lowering does not allocate a missing local: resolving a read requires that the root already exist, and the compiler is *checked*, returning an error for such a program.

Place-to-register compilation ($\Downarrow_{\text{CP}}$). The judgment $C \vdash_k p \Downarrow_{\text{CP}} (r, D, C')$ lowers place p , leaves a pointer to it in r , and returns a cleanup list. Its index k is the retag kind of any borrow the lowering emits: reads lower their place with $k = \text{shared}$ and assignment destinations with $k = \text{mutable}$. **PLACE-LOCAL** is a lookup. A zero-offset projection reuses the base pointer (**PLACE-PROJ-0**): it denotes the same address and needs no narrower pointer. A projection at a nonzero byte offset emits a borrow of the *final selected size* (**PLACE-PROJ-N**). A dereference loads the eight-byte pointer (**PLACE-DEREF**) and retires the borrows that were needed to reach it, but it returns an empty cleanup list: the loaded tag belongs to the source program and must not be retired.

Definition 2.1 (Route borrow) *A route borrow is the borrow instruction emitted by PLACE-PROJ-N to obtain a pointer to a projected subregion of a place that the compiled code already holds a pointer to. Its fresh tag is a route tag. A route borrow is unprotected, has an empty mask, and spans exactly the bytes of the layout selected by the place. Its register and size are recorded as an entry (r, n) of the cleanup list D , and the tag is retired by $\text{die}(r, n)$ before the next source-statement boundary. A route tag is never stored to memory by the compiled code and has no source counterpart.*

Route tags are the only tags the compiled code mints that the source does not, apart from the borrow that **RHS-MOVE** retires within the same statement, as the source does. Every other target retag corresponds to a source `ref` and produces a tag that the source program also stores.

Reassociation. A place may contain nested projections, but emitting a borrow for each syntactic layer would change the accessed ranges. **PLACE-ASSOC** reassociates adjacent paths before any borrow is emitted: for $q : \sigma_0 \rightarrow \sigma$ and $p : \sigma \rightarrow \tau$ the composed path $q \cdot p : \sigma_0 \rightarrow \tau$ selects the same field, and $\text{off}_\Lambda(b, q \cdot p) = \text{off}_\Lambda(b, q) + \text{off}_\Lambda(b, q, p)$. Reassociation stops at a dereference, because an offset cannot be moved across a memory-loaded pointer. For x.1.0, lowering without **PLACE-ASSOC**

Table 6. Compilation rules. $\text{root}(d)$ is the local reached from d through projections and dereferences. In **PLACE-PROJ-0** and **PLACE-PROJ-N** the base b is not itself a projection (**PLACE-ASSOC** applies first). $\beta_d = \Lambda(d)$ is the destination's layout; **RHS-REF** passes $m' = \text{bytes}_{\Lambda(p)}(m)$.

Premises	Conclusion	Rule Name
$\text{root}(d) = \ell, L(\ell) \text{ defined}$	$C \vdash \text{root}(d) \rightarrow C$	ROOT-BOUND
$\text{root}(d) = \ell : \tau, L(\ell) = \perp, r \text{ fresh in } C$	$C \vdash \text{root}(d) \rightarrow (C \oplus [r := \text{alloc}_{\Lambda(\ell)}])[L(\ell) \mapsto (r, \tau)]$	ROOT-ALLOC
$L(\ell) = (r, \tau)$	$C \vdash_k \ell \Downarrow_{\text{CP}} (r, [], C)$	PLACE-LOCAL
$C \vdash_k b.(q \cdot p) \Downarrow_{\text{CP}} (r, D, C')$	$C \vdash_k (b.q).p \Downarrow_{\text{CP}} (r, D, C')$	PLACE-ASSOC
$\text{off}_{\Lambda}(b, q) = 0, C \vdash_k b \Downarrow_{\text{CP}} (r_b, D_b, C')$	$C \vdash_k b.q \Downarrow_{\text{CP}} (r_b, D_b, C')$	PLACE-PROJ-0
$\delta = \text{off}_{\Lambda}(b, q) > 0, n = \Lambda(b.q) ,$ $C \vdash_k b \Downarrow_{\text{CP}} (r_b, D_b, C_1), r_f \text{ fresh in } C_1,$ $I = (r_f := \text{borrow}(k, \text{false}, [], n, r_b, \delta))$	$C \vdash_k b.q \Downarrow_{\text{CP}} (r_f, D_b \oplus [(r_f, n)], C_1 \oplus [I])$	PLACE-PROJ-N
$C \vdash_{\text{shared}} p \Downarrow_{\text{CP}} (r_p, D_p, C_1), r \text{ fresh in } C_1$	$C \vdash_k *p \Downarrow_{\text{CP}} (r, [], C_1 \oplus [r := \text{load}_{\text{ptr}(\Lambda(*p))}(r_p)] \oplus \text{cleanup}(D_p))$	PLACE-DEREF
$L(\ell) = (r_b, \tau), r \text{ fresh in } C$	$C \vdash_{k,c,m} \ell \Downarrow_{\text{CB}} (r, C \oplus [r := \text{borrow}(k, c, m, \Lambda(\ell) , r_b, 0)])$	BPLACE-LOCAL
$C \vdash_k b \Downarrow_{\text{CP}} (r_b, D_b, C_1), r \text{ fresh in } C_1$	$C \vdash_{k,c,m} b.q \Downarrow_{\text{CB}} (r, C_1 \oplus [r := \text{borrow}(k, c, m, \Lambda(b.q) , r_b, \text{off}_{\Lambda}(b, q)])]$	BPLACE-PROJ
$C \vdash_{\text{shared}} p \Downarrow_{\text{CP}} (r_p, D_p, C_1),$ $r_l, r \text{ fresh in } C_1$	$C \vdash_{k,c,m} *p \Downarrow_{\text{CB}} (r, C_1 \oplus [r_l := \text{load}_{\text{ptr}(\Lambda(*p))}(r_p)] \oplus \text{cleanup}(D_p) \oplus [r := \text{borrow}(k, c, m, \Lambda(*p) , r_l, 0)])]$	BPLACE-DEREF
	$C \vdash \text{const}(w) \Downarrow_{\text{CR}} (\lambda r_d. [\text{store}_{\beta_d}([\text{dat}(w)], r_d)], C)$	RHS-CONST
$C \vdash_{\text{shared}} p \Downarrow_{\text{CP}} (r_s, D_s, C_1), r_v \text{ fresh in } C_1$	$C \vdash \text{copy}(p) \Downarrow_{\text{CR}} (\lambda r_d. [\text{store}_{\beta_d}(r_v, r_d)], C_1 \oplus [r_v := \text{load}_{\Lambda(p)}(r_s)] \oplus \text{cleanup}(D_s))$	RHS-COPY
$C \vdash_{\text{mutable}, \text{false}, []} p \Downarrow_{\text{CB}} (r_m, C_1),$ $r_v \text{ fresh in } C_1$	$C \vdash \text{move}(p) \Downarrow_{\text{CR}} (\lambda r_d. [\text{store}_{\beta_d}(r_v, r_d)], C_1 \oplus [r_v := \text{load}_{\Lambda(p)}(r_m), \text{die}(r_m, \Lambda(p))])$	RHS-MOVE
$C \vdash_{k,c,m'} p \Downarrow_{\text{CB}} (r_v, C_1)$	$C \vdash \text{ref}(k, c, m, p) \Downarrow_{\text{CR}} (\lambda r_d. [\text{store}_{\beta_d}(r_v, r_d)], C_1)$	RHS-REF
$C \vdash \text{root}(d) \rightarrow C_1, C_1 \vdash e \Downarrow_{\text{CR}} (F, C_2),$ $C_2 \vdash_{\text{mutable}} d \Downarrow_{\text{CP}} (r_d, D_d, C_3)$	$C \vdash d := e \rightarrow_c C_3 \oplus F(r_d) \oplus \text{cleanup}(D_d)$	STMT-ASSIGN
	$C \vdash \text{halt} \rightarrow_c C \oplus [\text{halt}]$	STMT-HALT
	$C \vdash [] \rightarrow_p C$	PROG-EMPTY
$C \vdash s \rightarrow_c C_1, C_1 \vdash P \rightarrow_p C'$	$C \vdash s :: P \rightarrow_p C'$	PROG-STEP

would borrow the sixteen-byte field $x.1$ and then reuse its zero-offset subfield, retagging [16, 32]. With it, the compiler emits one eight-byte borrow at the composed offset 8 and leaves the sibling $x.1.1$ alone.

Escaping borrows ($\Downarrow_{\text{CB}}$). Reference formation needs the distinct judgment $C \vdash_{k,c,m} p \Downarrow_{\text{CB}} (r, C')$. It follows the same path reassociation but *always* ends in a $\text{borrow}(k, c, m, |\beta|, \dots)$, including at offset zero (**BPLACE-LOCAL**, **BPLACE-PROJ**), and for a dereference it first loads the stored

pointer and then borrows the pointed-to range (**BPLACE-DEREF**). Its final tag is not a route tag and is never retired: it escapes in the pointer value the program stores, and the correctness proof extends the source-to-target tag map with it. The mask a reference carries is the source's, expanded to one entry per byte.

Expression-to-instructions compilation ($\Downarrow_{\text{CR}}$). The judgment $C \vdash e \Downarrow_{\text{CR}} (F, C')$ emits all the work that precedes the destination and returns a *store function* F that awaits the destination register. **RHS-CONST** emits nothing. **RHS-COPY** materializes the copied values in a register *before* the destination is lowered. This is what realizes the source rule's order, evaluate, then resolve the destination, then write, when both resolutions have observable permission effects. **RHS-MOVE** does the same through a mutable borrow that it retires after the load, the events of **E-MOVE**. **RHS-REF** retains the escaping tag. Every store is at the destination's layout β_d , as the source writes at it. Every expression of the full language has the same shape (Appendix A).

Compilation judgments ($\rightarrow_c, \rightarrow_p$). **STMT-ASSIGN** is the only assignment rule, for every destination: the emitted intervals concatenate as

$$\text{root}(d); \text{ pre-code of } e; \text{ lowering of } d; F(r_d); \text{ cleanup}(D_d).$$

PROG-EMPTY and **PROG-STEP** fold statement compilation from left to right from the initial state $C_{\text{init}} = (0, 0, \emptyset, \emptyset)$; the target program is the final code map, or the first compiler error. For source statement index i , compiling the prefix $P[0..i]$ from C_0 produces a state C_i whose next label $C_i.n_l$ is the target entry label of statement i . Monotonicity gives the two facts simulation needs,

$$C_i.n_l \leq C_{i+1}.n_l \quad \wedge \quad \forall j < C_i.n_l. C_{i+1}.K(j) = C_i.K(j),$$

so compiling later statements cannot change a fragment already emitted. No fixed instruction-per-statement ratio is assumed.

Example. We compile the running program from C_{init} (Fig. 4), tracking (n_r, n_l, L) . Statement 0: $L(x) = \perp$, so **ROOT-ALLOC** emits label 0, an allocation at $\beta_x = \Lambda(x)$, and sets $L(x) = (R_0, \tau_x)$; **RHS-CONST** has no pre-code; $x.0$ has byte offset zero, so **PLACE-PROJ-0** and **PLACE-LOCAL** return $(R_0, [])$ and emit nothing; $F(R_0)$ is label 1, a store at the destination's layout $\text{int}(8)$. The state is $(1, 2, \{x \mapsto R_0\})$. Statement 1: **ROOT-BOUND**; **PLACE-ASSOC** rewrites $(x.q_1).q_0$ to $x.(q_1 \cdot q_0)$ with byte

MIRLITE	OSEA-IR
0: $x.0 := \text{const}(5)$	0: $R_0 := \text{alloc}_{\beta_x}$ 1: $\text{store}_{\text{i8}}([\text{dat}(5)], R_0)$
1: $x.1.0 := \text{const}(42)$	2: $R_1 := \text{borrow}(\text{mutable}, \text{false}, [], 8, R_0, 8)$ 3: $\text{store}_{\text{i8}}([\text{dat}(42)], R_1)$ 4: $\text{die}(R_1, 8)$
2: $y := \text{ref}(\text{mutable}, \text{false}, [], x.1.0)$	5: $R_2 := \text{alloc}_{\text{ptr}(\text{i8})}$ 6: $R_3 := \text{borrow}(\text{mutable}, \text{false}, [0]^8, 8, R_0, 8)$ 7: $\text{store}_{\text{ptr}(\text{i8})}(R_3, R_2)$
3: $*y := \text{const}(7)$	8: $R_4 := \text{load}_{\text{ptr}(\text{i8})}(R_2)$ 9: $\text{store}_{\text{i8}}([\text{dat}(7)], R_4)$
4: halt	10: halt

Fig. 4. Compilation of the running program. Each row is one source statement and the labelled instructions emitted for it; i8 abbreviates $\text{int}(8)$, and $[0]^8$ is the mask that marks none of the eight bytes.

offset $8 + 0 = 8$ and selected size $|\text{int}(8)| = 8$; **PLACE-PROJ-N** emits the route borrow at label 2 and returns $(R_1, [(R_1, 8)])$; the store is label 3; the cleanup is label

(4) Every number is derived: the layout gives the offset and the borrow size, the destination's layout the store, and reversing the singleton cleanup list gives the die. Statement 2: **ROOT-ALLOC** emits label 5 for y before the expression, mirroring **PREP-ALLOC**; **RHS-REF** uses **BPLACE-PROJ**, whose borrow at label 6 has the shape of the route borrow of label 2, with the source's empty mask expanded to eight bytes, but is recorded in no cleanup list; label 7 stores it. There is no die. Statement 3: **PLACE-DEREF** lowers y by **PLACE-LOCAL**, emits the eight-byte pointer load at label 8, and returns $(R_4, [])$; label 9 stores through the loaded pointer. There is again no die: retiring R_4 's tag would pop the program's own reference. The final state is $(5, 11, \{x \mapsto R_0, y \mapsto R_2\})$, and its code map is the program that Table 5 executes.

Formalization of compiler state growth, checked lowering, and program compilation: `src/obseq3/compile.lean`. The listing of Fig. 4 is the witness `g14_paper_running_example` in `src/obseq3/compile_tests.lean`.

2.5 Correctness

Correctness is a forward simulation of successful executions. Literal state equality is impossible: the target has registers and introduces route tags (Definition 2.1). The relation instead compares source-observable memory, local pointers, and permission stacks at source-statement boundaries. This section defines exactly the machinery needed to state that relation (Definition 2.2 to Definition 2.7), states the simulation theorems (Theorem 2.8 to Corollary 2.11), and reads the running program as a simulation (Table 7). The proofs are mechanized (Section 3) and are not reproduced here.

2.5.1 Renaming and the byte relations.

Definition 2.2 (Tag renaming) *A tag renaming is a partial map $\rho_t : \text{Tag} \rightarrow \text{option}(\text{Tag})$. A renaming ρ'_t extends ρ_t , written $\rho_t \subseteq \rho'_t$, when ρ'_t agrees with ρ_t wherever ρ_t is defined.*

The map is partial because future tags have not yet been created. Tags genuinely require renaming: route borrows advance the target tag counter, so a later source and target ref may mint different numeric tags. Addresses, by contrast, are not renamed at all. Each machine allocates with the same deterministic bump allocator, which never reuses an address, and both start at the same watermark. The compiler emits exactly one target allocation, of the same layout, for each source allocation, and in the same order. Hence the allocators stay in lockstep (I4 below), and every block has the same base, and every byte the same address, on both sides.

Definition 2.3 (Byte and memory simulation) *Under ρ_t , a source byte α is simulated by a target byte α' , written $\alpha \approx \alpha'$, when $\alpha = \text{uninit}$, or $\alpha = \text{init}(x, \pi)$ and $\alpha' = \text{init}(x, \pi')$ where either $\pi = \pi' = \perp$ or $\pi = (b, sz, e, t)$ and $\pi' = (b, sz, e, t')$ with $\rho_t(t) = t'$. Source memory μ_s is simulated by target memory μ_t when $\mu_s(a) \approx \mu_t(a)$ at every address a , and the two allocators are in lockstep when μ_s and μ_t have the same allocation table, the same watermark, and the same freed bases.*

The relation is forward only: an uninitialized source byte refines any target byte, because a successful source run cannot observe it, and the target may hold bytes the source never wrote. An initialized byte is the same byte on both sides; only the tag of its provenance is renamed. Values inherit the relation through decoding:

Definition 2.4 (Value simulation) *Under ρ_t , a source value v is simulated by a target value v' , written $v \approx v'$, when $v = \text{word}(w)$ and $v' = \text{dat}(w)$; or $v = \text{ptr}(b, o, e, sz, t)$ and $v' = \text{ptr}(b, o, e, sz, t')$ with $\rho_t(t) = t'$; or $v = \text{undef}$.*

Related bytes decode to related values at every scalar, and a stored value related to the target's encodes to related bytes whenever both encodings succeed; this is what lets a load and a store move a value between the two memories through any byte layout, including a one-byte field inside a wider one.

2.5.2 Permission simulation.

Definition 2.5 (Permission simulation) *Let items, stacks, and states be those of the Stacked Borrows instance of Section 2.1. Under ρ_t :*

- Two items are related when they have the same constructor and ρ_t maps the source tag to the target tag; raw-pointer items additionally carry the same mutability flag.
- Two stacks are related position by position, so their lengths agree.
- Two stack maps are related when, for every byte address a , either both lack an entry at a or their entries at a are related stacks.
- Two protector frame lists are related as lists of related tag lists, and two exposed-tag lists, and two lists of weakly protected tags, are related as tag lists.

Permission states Π_s and Π_t are related, written $\Pi_s \approx \Pi_t$, when all of these components are related and $\Pi_s.\text{NextTag} \leq \Pi_t.\text{NextTag}$.

Stack-map comparison is by address lookup, not by the list position used to implement the finite map. Disjoint operations may reorder that representation while leaving all address observations the same.

The counter condition is an inequality rather than an equality. Suppose a route borrow mints route tag u at the target counter. Inside the compiled fragment the target stacks contain an extra item, so the boundary relation is not expected to hold. The store uses u and die removes its item. The target counter remains advanced, which is why a later pair of corresponding source and target tags may have different numeric values.

When a source-visible reference is formed, both machines mint a tag. If they produce t_s and t_t , the proof extends

$$\rho'_t = \rho_t[t_s \mapsto t_t].$$

Because both fresh tags are minted at their machine's counter, and the boundary invariant below keeps every mapped tag strictly below both counters, neither fresh tag is already in the old map's domain or range, so injectivity is preserved. By contrast, a route borrow never extends ρ_t , since there is no source tag to use as a key.

2.5.3 The layout conditions.

Definition 2.6 (Layout conditions) *A layout environment Λ is well formed when every pointer-typed place has an eight-byte layout, and every integer- or pointer-typed place p has a layout whose first leaf spans it: $|\kappa_0| = |\Lambda(p)|$ for the scalar κ_0 of the first leaf of $\Lambda(p)$. Each local's layout has its type's shape when $\text{Int } \theta$ is laid out as $\text{int}(n)$ with n the byte width of θ , $\text{Ptr } \tau$ as $\text{ptr}(\beta)$ with β of τ 's shape, and a tuple as tup with fields of their types' shapes, offsets, size, and alignment unconstrained.*

The first condition is what the compiler relies on when it loads a pointer in eight bytes through a borrow of the pointer's field; the second, when it reads one leaf of a field at a nonzero offset

through a borrow of the field's size. Neither condition is about Rust: they say that the layout table is a layout *of the program's types*. Shape agreement is decidable per local, implies well-formedness for every place, and holds for the uniform layout (every integer eight bytes, tuples in C layout) and, by a check the conformance runner performs, for every program it loads with rustc's layouts (Section 3).

2.5.4 The boundary invariant.

Definition 2.7 (Boundary invariant) *Let C be a compiler state. Source state $S = (i, E, \mu_s, \Pi_s)$ and target state $T = (j, R, \mu_t, \Pi_t)$ satisfy the boundary invariant at C under ρ_t , written $S \approx_{\rho_t}^C T$, when the following nine clauses hold.*

- (I1) Control alignment. $j = C.n_l$.
- (I2) Bound locals. If $E(\ell) = (a, t)$ for $\ell : \tau$, then $C.L(\ell) = (r, \tau)$, $R(r) = [\text{ptr}(a, 0, e, |\Lambda(\ell)|, t')]$ for some e , $\rho_t(t) = t'$, and t is not the wildcard tag.
- (I3) Memory. μ_s is simulated by μ_t (Definition 2.3).
- (I4) Allocator lockstep. The two allocators are in lockstep (Definition 2.3).
- (I5) Permissions. $\Pi_s \approx \Pi_t$ (Definition 2.5).
- (I6) Tag-map shape. ρ_t is injective and maps the wildcard tag to itself.
- (I7) Tag bounds. Every source tag in the domain of ρ_t lies below $\Pi_s.\text{NextTag}$, and every tag in its range lies below $\Pi_t.\text{NextTag}$.
- (I8) Unbound locals. If E has no binding for ℓ , then $C.L$ has no entry for ℓ .
- (I9) Register freshness. Every register in the range of $C.L$ is below $C.n_r$.

The invariant is required only at source-statement boundaries and is deliberately strict about stack shape there. The compiled fragment may push a route tag's item inside a fragment, but die removes it before the invariant is re-established. Numeric next-tag counters may still differ, which is why tags are renamed and the target counter need only be greater. For a program P , the boundary that matters after i source steps is the *prefix state* C_i , the result of compiling $P[0..i]$ from C_{init} ; (I1) then places the target at the entry label of statement i .

Each clause beyond (I3) and (I5) is there because a statement of the theorem without it is false or unprovable for some compiled fragment: memory agreement alone cannot show that a fresh load register does not overwrite a local's register (I9), and equal watermarks alone cannot show that an unbound source local corresponds to a target fragment containing `alloc` (I8).

2.5.5 Simulation theorems.

Let `srcStep` fetch the source statement at the current PC and execute it, and `runSn` and `runTn` iterate the two machines, treating `halt` and a missing statement as successful fixed points. The theorems hold for *every* statement and expression of the full language (Appendix A).

Theorem 2.8 (One-step forward simulation) *Let Λ be well formed (Definition 2.6), $s \neq \text{halt}$ a statement, C a compiler state from which s compiles to C' , and Q a program that contains the code of C' . If*

$$S \approx_{\rho_t}^C T \quad \text{and} \quad \text{step}(S, s) = \text{ok}(S'),$$

then there exist $\rho'_t \supseteq \rho_t$, a target state T' , and n_t such that

$$Q \vdash T \xrightarrow{n_t} T' \quad \text{and} \quad S' \approx_{\rho'_t}^{C'} T'.$$

The compiler state after the step is C' , the state compiling s left: the invariant moves along the compilation as the source moves along the program. Only the renaming grows. The target step count n_t is existential because fragments have different lengths.

The initial states are S_{init} and T_{init} : empty environment, register file, and memory, PC zero, both watermarks at 1, and both permission states initialized. They satisfy $S_{\text{init}} \approx_{\rho_0}^{C_{\text{init}}} T_{\text{init}}$ under the renaming ρ_0 that maps only the wildcard tag to itself.

Theorem 2.9 (Compiler correctness) *Let Λ be well formed and P a program that compiles to Q . If $\text{run}^{S^s}(P, S_{\text{init}}) = \text{ok}(S')$, then there exist ρ_t, T' , and n_t with*

$$\text{run}^{T^{n_t}}(Q, T_{\text{init}}) = \text{ok}(T') \quad \text{and} \quad S' \approx_{\rho_t}^{C_{S'^i}} T'.$$

Corollary 2.10 (Layouts of the program's types) *Theorem 2.9 holds for every Λ in which each local's layout has its type's shape.*

Corollary 2.11 (Uniform layout) *Theorem 2.9 holds, with no hypothesis beyond compilation, for the uniform layout.*

These results are preservation of successful finite executions. They are not backward simulation, divergence preservation, or an equivalence between source and target error messages. Their observable consequences are the memory and permission clauses of the final invariant, made explicit in Corollary 2.13.

2.5.6 The running program as a simulation.

Table 7. The running program as a simulation: the boundary invariant at each source-statement boundary. i is the source PC and $j = C_i.n_i$ the target label (Fig. 4); the ρ_t column shows what the step *adds*; “next” is the pair of tag counters.

i	j	added to ρ_t	next	what the invariant records at this boundary
0	0	$0 \mapsto 0$	1, 1	Initial states: nothing is bound, and ρ_t fixes only the wildcard.
1	2	$1 \mapsto 1$	2, 2	x is bound on both sides (I2): the allocators in lockstep (I4) gave both allocations base 8, and the two owning tags form the pair (1, 1).
2	5	—	2, 3	The stacks of [16, 24) are again related (I5): borrow; storec; die left the stack effect of the source's one useMut. Route tag 2 is in no renaming; the counters part.
3	8	$2 \mapsto 3,$ $3 \mapsto 4$	4, 5	y is bound, and the bytes of the reference it holds are related (I3): the same address bytes, with provenance tags 3 and 4. ρ_t grew by two pairs that are <i>not</i> numerically equal, which (I6) and (I7) permit.
4	10	—	4, 5	[16, 24) holds the same bytes on both sides (I3): the loaded ptr(8, 8, 8, 24, 3) and ptr(8, 8, 8, 24, 4) were related, so both writes went through related tags to the same bytes.

Example 2.12 Table 7 replays Table 3 against Table 5. Each row is an instance of the invariant that Theorem 2.9 asserts at the prefix state; the first is the initial relation and each later one follows from the previous by Theorem 2.8.

Boundary 1. Statement 0 assigns to the unbound local x . Source preparation allocates $|\beta_x| = 24$ bytes and obtains the owning tag 1; the target fragment begins with $R_0 := \text{alloc}_{\beta_x}$, allocates the same base by (I4), and obtains tag 1. The proof extends ρ_t by $1 \mapsto 1$ and the local map by $x \mapsto (R_0, \tau_x)$; the store writes the same eight bytes on both sides, establishing (I3) for $[8, 16]$.

Boundary 2. Statement 1 is $x.1.0 := \text{const}(42)$. The source resolves the place to address 16 and performs $\text{useMut}(\Pi_s, 16, 8, 1)$. The target performs

$$\text{ref}(\Pi_t, 16, 8, 1, \text{mutable}, \text{false}, []) = (\Pi_1, 2), \quad \text{useMut}(\Pi_1, 16, 8, 2) = \Pi_2, \quad \text{die}(\Pi_2, 16, 8, 2) = \Pi'_t.$$

The third event removes the route tag's item from each of the eight stacks, so they are again related (I5) under the *unchanged* ρ_t : tag 2 has no source counterpart and never escapes. Both memories update only $[16, 24]$, with the same bytes, so (I3) is restored and the neighbouring bytes are framed. (I1) advances from source PC 1 to 2 and from target label 2 to the next prefix label 5. The target counter is now 3 and the source counter 2, which the inequality of Definition 2.5 absorbs.

Boundary 3. Statement 2 allocates y on both sides, with owning tags 2 and 3, and then forms the reference, with tags 3 and 4. Neither pair is an equality, and neither could be: the route borrow of the previous statement advanced only the target counter. This is the step at which a tag renaming, rather than tag equality, is forced. The stored pointers $\text{ptr}(8, 8, 8, 24, 3)$ and $\text{ptr}(8, 8, 8, 24, 4)$ are related by Definition 2.4, so their encodings, the same eight address bytes with provenance tags 3 and 4, are related byte by byte (Definition 2.3); and the stacks of $[16, 24]$, $[\text{MutRef}(3), \text{Own}(1)]$ and $[\text{MutRef}(4), \text{Own}(1)]$, are related by Definition 2.5.

Boundary 4. Statement 3 writes through y . Both machines read $[32, 40]$ through y 's owning tag, related by $2 \mapsto 3$, and decode pointers related by (I3). The two writes therefore go through tags related by $3 \mapsto 4$, to the same bytes. No tag is minted and none is retired.

Reassociation is semantically load-bearing at boundary 2. A route borrow of the intermediate field $x.1$ would retag $[24, 32]$ as well, so the target would not simulate the source's eight-byte write in the presence of a borrow of the sibling $x.1.1$. The compiled path must use the composed offset and the final size.

The four formal views of the program thus line up. MIRLite's typed path selects $x.1.0$, which the layout places at byte offset 8 with size 8. OSEA-IR begins with a pointer to x 's 24 bytes in R_0 , which the compiler's local map connects to $x : \tau_x$. Reassociation prevents an intermediate sixteen-byte borrow, so place lowering creates exactly one route tag for $[16, 24]$, and cleanup supplies the matching eight-byte die. At each boundary, the shared addresses relate the updated bytes, memory simulation frames their neighbours, permission simulation cancels the route tag without adding it to ρ_t , and prefix compilation places the target at the next fragment exactly when the source reaches the next statement:

typed source range $\rightarrow$ reassociated target route $\rightarrow$ explicit events $\rightarrow$ boundary simulation.

2.5.7 Observable consequences.

Corollary 2.13 (Observations at a boundary) *If $S \approx_{\rho_t}^C T$, then:*

- every initialized source byte is the same byte in the target memory, at the same address, with the same provenance up to the tag;
- hence a leaf that decodes to $\text{word}(w)$ in the source decodes to $\text{word}(w)$ in the target, and one that decodes to $\text{ptr}(b, o, e, sz, t)$ decodes to $\text{ptr}(b, o, e, sz, t')$ with $\rho_t(t) = t'$;
- if $E(\ell) = (a, t)$ for $\ell : \tau$, then the register $C.L(\ell)$ holds exactly one pointer with base a , offset zero, size $|\Lambda(\ell)|$, and tag $\rho_t(t)$;
- the target stack at every byte address has the same length and item constructors as the source stack, with every source tag renamed by ρ_t ;
- the target PC is the entry label $C.n_l$.

Several asymmetries are intentional. The target may contain extra registers, because value temporaries remain after use. An uninitialized source byte refines any target byte, so the memory relation has no reverse guarantee. Tag counters need not be equal, and numeric tag equality is not observable. Finally, the invariant is required only at source-statement boundaries; intermediate target states may contain live route tags.

These choices are the minimum abstraction needed by the actual generated code. Strengthening to literal register equality would reject harmless temporaries, while weakening permission stacks to ignore arbitrary target items would no longer justify later accesses.

3 Mechanization and scope

PRECISE CLAIM The mechanized theorems are forward simulations for successful finite runs of every MIRLite program, every statement and expression of Appendix A included, on byte-addressed memory, for every layout environment that is well formed (Definition 2.6). Well-formedness holds outright for the uniform layout, and follows from a decidable per-local check that the conformance runner applies to the layouts it loads from rustc.

Every numbered definition and result of Section 2.5 is a Lean 4 declaration in `src/obseq3/proof/`:

Table 8. The results of Section 2.5 and their Lean declarations.

This paper	Lean declaration	File
Definition 2.2	TagRenameMap, TagRenameIncr	basis.lean
Definition 2.3	ByteSim, ByteMemSim, ByteAllocLockstep	memsim.lean
Definition 2.4	ValSim	memsim.lean
Definition 2.5	PermSim	basis.lean
Definition 2.6	PtrPlacesWF, LeafWF, LocalsAgree	places.lean, leaffield.lean; layoutagree.lean
Definition 2.7	InvAtB	spine.lean
Theorem 2.8	stmt_sim	coverage.lean
initial relation	InvAtB_initial	program.lean
Theorem 2.9	compile_correct_all	coverage.lean
Corollary 2.10	compile_correct_agrees	layoutagree.lean
Corollary 2.11	compile_correct_uniform	coverage.lean

The proof is about 12,900 lines and 410 theorems across the 36 files of that directory, which also hold the Stacked Borrows lemmas it rests on. None of it contains an admitted goal. A checked audit prints the axioms that Theorem 2.9 and its corollaries depend on and fails if that set differs in either direction from a pinned whitelist; a second check covers every one of the directory’s 780 declarations. The whitelist contains exactly the three standard Lean axioms, propositional extensionality, choice, and quotient soundness, and no `sorryAx`.

The executable compiler is additionally validated by testing: a compiler witness corpus of 138 programs, run on both machines at the uniform layout and pinned as golden listings where the shape of the code matters; a corpus of 195 entries, drawn from Miri’s Stacked Borrows tests and completed by local witnesses, loaded from rustc’s MIR through Charon with rustc’s own layouts, whose 176 supported programs reach Miri’s verdict and, where Miri reports undefined behavior, the same statement and, with four documented exceptions, the same reason; and a differential run that compiles each of those 176 programs and requires the same verdict from both machines. The layout check of Definition 2.6 passes on all of them. The running program of this paper is part of the witness corpus, both as a golden listing (Fig. 4) and as a differential test; the states of Table 3 and Table 5 are printed by the mechanized interpreters.

```
Invariant and simulation vocabulary: src/obseq3/proof/basis.lean, memsim.lean, spine.lean. Statement
and whole-program forward simulations: src/obseq3/proof/program.lean, coverage.lean. Witness corpus:
src/obseq3/compile_tests.lean; state dump of the running program: notes/2026-09-18-paper-running-
example.lean.
```

A The full executable surface

The main text presents constants, copies, moves, references, assignment, and `halt`. This appendix extends each of its figures and tables, in the same format, to the language the compiler and the theorem actually cover.

A.1 Syntax

Typing follows Section 2.2: `uninit` has any type; `alloc(len)` and the pointer expressions have pointer types, with `ptrCast`, `ptrOffset`, `refSlice`, `sliceLen`, and `subSlice` taking a pointer place;

MIRLITE	OSEA-IR
$\text{Expr} \ni e ::= \dots$ <code>uninit</code> <code>alloc(len)</code> <code>exposeAddr(p)</code> <code>addr(p)</code> <code>fromExposed(p)</code> <code>ptrCast(p)</code> <code>ptrOffset(p, d, i)</code> <code>refSlice(k, c, p)</code> <code>sliceLen(p)</code> <code>subSlice(p, p_l, p_h)</code> <code>binOp(op, p_a, p_b)</code> $\text{Stmnt} \ni s ::= \dots$ <code>assignIf(p = w, d := e)</code> <code>dealloc(p)</code> <code>pushProtectors</code> <code>popProtectors</code> $\text{len} ::= \text{const}(n)$ <code>from(p)</code>	$\text{Rhs} \ni h ::= \dots$ <code>allocN_β(n)</code> <code>allocDyn_β(r)</code> <code>expose_κ(r)</code> <code>fromExposed_κ(r)</code> <code>offset_κ(r, δ, i)</code> <code>borrow(k, c, m, ⊥, r, δ)</code> <code>sliceLen_n(r)</code> <code>subSlice_n(r, r_l, r_h)</code> <code>binOp(op, r_a, r_b)</code> $\text{Instr} \ni I ::= \dots$ <code>dealloc(r)</code> <code>skipIf(r, w, n)</code> <code>pushProt</code> <code>popProt</code>

Fig. 5. The remaining syntax of MIRLite (left) and OSEA-IR (right), extending Fig. 2 and Fig. 3. d in `ptrOffset` and δ in `offset` are integers and i a boolean, set for in-bounds arithmetic; a borrow of length $\perp$ retags the pointer’s extent; `op` is an integer operation at an integer type.

exposeAddr(p), addr(p), sliceLen(p), and binOp have any integer type, the destination's; integer operands, slice bounds, allocation lengths, and guard discriminants are integer places of any width. An integer operation carries its own integer type: it wraps at that width, compares by its signedness, and is undefined behavior on overflow for the unchecked forms and on division by zero. An integer cast of rustc's MIR is lowered by the loader to such an operation at the destination's type.

The source semantics of these forms follows the pattern of Table 2: each expression resolves its place, performs the permission events listed for its target counterpart in Table 10 in the same order, and produces values. The one-leaf expressions, exposeAddr, addr, fromExposed, ptrCast, ptrOffset, and refSlice, read the first leaf of their place, of scalar κ , rather than the whole place. addr decodes that leaf's bytes at integer type, as a pointer-to-integer transmute or ptr. addr() does: the address, with the provenance stripped and nothing exposed, so a pointer later rebuilt from it by fromExposed may not access the allocation unless some other cast exposed it. A guarded assignment first prepares the root of its destination, *on both paths*, then reads its discriminant exactly as copy does, a real read access, and performs the assignment when the value read equals w .

A.2 The permission model

Table 9. The remaining operations of the permission interface of Section 2.1.

Operation	Meaning
dealloc(Π, a, n, t) = Π'	Deallocate the n bytes from a through t and forget their permissions. The item carrying t must grant writes at every byte, and no item of those stacks may be strongly protected.
expose(Π, t) = Π'	Record t as exposed, so that a later wildcard access may resolve to it.
pushProt(Π) = Π' , popProt(Π) = Π'	Open a protector frame; close the innermost frame, ending the protection of every tag registered in it.

Tags are drawn from a countable set with a distinguished *wildcard* tag 0, which stands for an unknown provenance recovered from an exposed integer. Every fresh tag is minted from NextTag upward, so it is distinct from all earlier ones and a tag minted later is numerically larger. The rules of Table 1 extend as follows. *Protectors*: if the flag c of a ref is set, the fresh tag is registered in the innermost protector frame; **SB-READ**, **SB-USE-MUT**, and **SB-DIE** fail when an item they would disable or remove is protected. A Box passed to a function is *weakly* protected: it may be deallocated through its own tag or one derived from it. *Masks*: entry i of the mask m says whether byte $a + i$ of the retagged range lies inside an UnsafeCell; a byte beyond the end of m counts as unmarked. On the bytes that m marks, a shared or raw-constant retag performs no access and inserts RawPtr(true, u) directly above the item carrying t , as **SB-REF-RAW** does. The mask has one entry per byte, rather than being a single flag, because one reference may cover both kinds of memory: in &(i32, Cell<i32>) the first field is frozen and the second is not, so a single retag pushes Ref(u) on the first four bytes and inserts RawPtr(true, u) on the next four. *Two-phase*: a two-phase retag performs a read and inserts RawPtr(true, u) likewise, modelling a reservation that stays writable until activation. *Raw-pointer groups*: **SB-USE-MUT** also accepts ι = RawPtr(true, t), and then keeps the contiguous run of mutable raw-pointer items directly above ι , removing only what lies above the whole group. *Wildcard*: an access through tag 0 resolves to the topmost exposed item that grants it. This rule is a determinization: for programs that use integer-to-pointer casts, the theorem is a statement about it rather than about Miri's angelic choice.

A.3 OSEA-IR

Table 10. The remaining OSEA-IR rules, extending Table 4. A *one-leaf read* through r at scalar κ means: $T.R(r) = [\text{ptr}(b, o, e, sz, t)]$, $a = b + o$, b live, $a + |\kappa| \leq b + sz$, $\text{read}(T.\Pi, a, |\kappa|, t) = \Pi_1$, and the leaf is $\text{dec}_\kappa(T.\mu[a..a + |\kappa|])$. *wild* is the wildcard tag and $\text{allocOf}(\mu, w)$ the allocated range containing address w , or $(w, 0)$.

Premises	Conclusion	Rule Name
$z = n \cdot \beta $, $\text{alloc}(T.\mu, z, \text{al}(\beta)) = (b, \mu')$, $\text{own}(T.\Pi, b, z) = (\Pi', u)$	$T \vdash \text{allocN}_\beta(n) \Downarrow_h$ $([\text{ptr}(b, 0, z, z, u)], T[\mu \mapsto \mu', \Pi \mapsto \Pi'])$	RVAL-ALLOCN
$T.R(r) = [\text{dat}(n)]$, and as RVAL-ALLOCN	$T \vdash \text{allocDyn}_\beta(r) \Downarrow_h$ $([\text{ptr}(b, 0, z, z, u)], T[\mu \mapsto \mu', \Pi \mapsto \Pi'])$	RVAL-ALLOCDYN
one-leaf read through r at κ , leaf = $\text{ptr}(b', o', e', sz', t')$	$T \vdash \text{expose}_\kappa(r) \Downarrow_h$ $([\text{dat}(b' + o')], T[\Pi \mapsto \text{expose}(\Pi_1, t')])$	RVAL-EXPOSE
one-leaf read through r at κ , leaf = $\text{word}(w)$, $\text{allocOf}(T.\mu, w) = (b', sz')$	$T \vdash \text{fromExposed}_\kappa(r) \Downarrow_h$ $([\text{ptr}(b', w - b', sz' - (w - b'), sz', \text{wild})], T[\Pi \mapsto \Pi_1])$	RVAL-FROMEXPOSED
one-leaf read through r at κ , leaf = $\text{ptr}(b', o', e', sz', t')$, $o' + \delta \geq 0$, $i \wedge \delta \neq 0 \Rightarrow b' \text{ live} \wedge o' \leq sz' \wedge o' + \delta \leq sz'$	$T \vdash \text{offset}_\kappa(r, \delta, i) \Downarrow_h$ $([\text{ptr}(b', o' + \delta, e', sz', t')], T[\Pi \mapsto \Pi_1])$	RVAL-OFFSET
$T.R(r) = [\text{ptr}(b, o, e, sz, t)]$, b live, $\text{ref}(T.\Pi, b + o + \delta, e, t, k, c, m) = (\Pi', u)$	$T \vdash \text{borrow}(k, c, m, \perp, r, \delta) \Downarrow_h$ $([\text{ptr}(b, o + \delta, e, sz, u)], T[\Pi \mapsto \Pi'])$	RVAL-BORROW-REST
$T.R(r) = [\text{ptr}(b, o, e, sz, t)]$	$T \vdash \text{sliceLen}_n(r) \Downarrow_h \ ([\text{dat}(e \div n)], T)$	RVAL-SLICELEN
$T.R(r) = [\text{ptr}(b, o, e, sz, t)]$, $T.R(\eta) = [\text{dat}(l)]$, $T.R(\eta_h) = [\text{dat}(h)]$, $l \leq h$, $h \cdot n \leq e$	$T \vdash \text{subSlice}_n(r, \eta, \eta_h) \Downarrow_h$ $([\text{ptr}(b, o + ln, (h - l)n, sz, t)], T)$	RVAL-SUBSLICE
$T.R(r_a) = [\text{dat}(x)]$, $T.R(r_b) = [\text{dat}(y)]$, op is defined on x, y	$T \vdash \text{binOp}(\text{op}, r_a, r_b) \Downarrow_h \ ([\text{dat}(\text{op}(x, y))], T)$	RVAL-BINOP
$Q(j) = \text{dealloc}(r)$, $R(r) = [\text{ptr}(b, 0, e, sz, t)]$, $\text{dealloc}(\Pi, b, sz, t) = \Pi'$	$(j, R, \mu, \Pi) \xrightarrow{\text{sea}} (j + 1, R, \mu[b..b + sz] \mapsto \text{uninit}, b \text{ freed}, \Pi')$	EXEC-DEALLOC
$Q(j) = \text{skipIf}(r, w, n)$, $R(r) = [\text{dat}(w')]$, $j' = j + 1$ if $w' = w$, else $j' = j + 1 + n$	$(j, R, \mu, \Pi) \xrightarrow{\text{sea}} (j', R, \mu, \Pi)$	EXEC-SKIPIF
$Q(j) = \text{pushProt}$	$(j, R, \mu, \Pi) \xrightarrow{\text{sea}} (j + 1, R, \mu, \text{pushProt}(\Pi))$	EXEC-PUSHPROT
$Q(j) = \text{popProt}$, $\text{popProt}(\Pi) = \Pi'$	$(j, R, \mu, \Pi) \xrightarrow{\text{sea}} (j + 1, R, \mu, \Pi')$	EXEC-POPProt

The pointer right-hand sides are operational rather than mere casts, and share a three-stage pattern: validate the register shape, perform the permission events in source order, construct a result. They read *through the outer pointer in the register* before inspecting the value in memory, one leaf at the scalar the source reads. For example, *offset* does not change the offset of that outer pointer: it loads a pointer-valued leaf, changes the loaded value, and returns the result in a register. This is why the compiler can use it to implement MIRLite pointer arithmetic without suppressing the source's read event. Its flag i is Miri's in-bounds arithmetic, set by the loader for *add*, *offset*, and a raw borrow of a field through a raw pointer, and *clear* for *wrapping_add* and *wrapping_offset*: a nonzero move must then stay within a live allocation, both ends included and one past the end allowed, so a pointer without provenance, whose allocation size is zero, cannot move. Both machines check the same condition on the same pointer fields and the same

freed blocks, as one shared function. `allocDyn` takes its length from a register that an ordinary load filled, which matches the source’s permission-visible length read. A deallocated block is overwritten with uninitialized bytes and its base is recorded as freed; the bump allocator never reuses it, and any later access through a pointer into it fails before its permissions are consulted, as in `Miri`. `skipIf` touches neither memory nor permissions: the discriminant it tests was loaded into a register by an ordinary load.

OSEA-IR errors fall into several semantic classes rather than one generic “stuck” state:

Table 11. Error classes of OSEA-IR.

Class	Typical cause	Where detected
Register shape	Missing register, or a register that does not hold the single pointer or word the instruction needs.	Before any memory or permission change.
Liveness	An access through a pointer into a freed block.	Before the bounds check and any permission event.
Spatial	A complete access range exceeds its allocation, a pointer offset becomes negative or, for in-bounds arithmetic, leaves its live allocation, or a sub-slice exceeds its extent.	Before the corresponding read/write permission event.
Permission	<code>read</code> , <code>ref</code> , <code>useMut</code> , <code>die</code> , <code>dealloc</code> , or a protector <code>pop</code> rejects the operation.	At the permission interface; its error is propagated.
Value	A load decodes an uninitialized leaf; a cast or allocation observes the wrong value constructor; a store’s word does not fit its leaf; an integer operation is undefined.	After any required permission read, so that event order remains explicit.

The forward simulation only starts from a successful source step. It therefore proves that the compiled fragment avoids all of these errors; it does not claim that source and target failure messages coincide.

A.4 Compiler

Every expression has the pre-code/store split of **RHS-COPY**, and every store is at the destination’s layout β_d . In Table 12, “lower p ” is $C \vdash p \Downarrow_{\text{CP}}^{\text{shared}} (r_s, D_s, C_1)$; “read p into r ” is “lower p ; $r := \text{load}_{\Lambda(p)}(r_s); \text{cleanup}(D_s)$ ”; r_v is a fresh value register; κ is the scalar of the first leaf of $\Lambda(p)$, and β_κ the layout of that one leaf; and the pre-code is emitted before the destination is lowered.

Table 12. Lowering of the remaining expressions, extending **RHS-CONST**, **RHS-COPY**, **RHS-MOVE**, and **RHS-REF** of Table 6. In `ptrOffset`, `sliceLen`, and `subSlice`, β_e is the pointee layout of p ; in `alloc`, β_e is the pointee of β_d .

Expression	Pre-destination code	Deferred store $F(r_d)$
<code>uninit</code>	None.	$\text{store}_{\beta_d}([\text{undef}]^n, r_d)$, n the leaves of β_d
<code>alloc(const(n))</code>	$r_v := \text{allocN}_{\beta_e}(n)$.	$\text{store}_{\beta_d}(r_v, r_d)$
<code>alloc(from(p))</code>	Read p into r_n ; $r_v := \text{allocDyn}_{\beta_e}(r_n)$.	$\text{store}_{\beta_d}(r_v, r_d)$
<code>exposeAddr(p)</code>	Lower p ; $r_v := \text{expose}_k(r_s)$; $\text{cleanup}(D_s)$.	$\text{store}_{\beta_d}(r_v, r_d)$
<code>addr(p)</code>	Lower p ; $r_v := \text{load}_{\text{int}(k)}(r_s)$; $\text{cleanup}(D_s)$.	$\text{store}_{\beta_d}(r_v, r_d)$
<code>fromExposed(p)</code>	Lower p ; $r_v := \text{fromExposed}_k(r_s)$; $\text{cleanup}(D_s)$.	$\text{store}_{\beta_d}(r_v, r_d)$
<code>ptrCast(p)</code>	Lower p ; $r_v := \text{load}_{\beta_e}(r_s)$; $\text{cleanup}(D_s)$.	$\text{store}_{\beta_d}(r_v, r_d)$
<code>ptrOffset(p, d, i)</code>	Lower p ; $r_v := \text{offset}_k(r_s, d \cdot \beta_e , i)$; $\text{cleanup}(D_s)$.	$\text{store}_{\beta_d}(r_v, r_d)$
<code>refSlice(k, c, p)</code>	Lower p ; $r_v := \text{load}_{\beta_e}(r_s)$; $\text{cleanup}(D_s)$; then $r_v := \text{borrow}(k, c, [], \perp, r_v, 0)$.	$\text{store}_{\beta_d}(r_v, r_d)$
<code>sliceLen(p)</code>	Read p into r ; $r_v := \text{sliceLen}_{ \beta_e }(r)$.	$\text{store}_{\beta_d}(r_v, r_d)$
<code>subSlice(p, p_l, p_h)</code>	Read p, p_l, p_h into r, r_l, r_h ; $r_v := \text{subSlice}_{ \beta_e }(r, r_l, r_h)$.	$\text{store}_{\beta_d}(r_v, r_d)$
<code>binOp(op, p_a, p_b)</code>	Read p_a, p_b into r_a, r_b ; $r_v := \text{binOp}(op, r_a, r_b)$.	$\text{store}_{\beta_d}(r_v, r_d)$

Pointer offsets are expressed by the source in pointee units, so the compiler scales d by the pointee's byte size before OSEA-IR sees it. A pointer cast is tag-preserving: a borrow would mint a new tag, whereas a load and a store copy the pointer value unchanged while performing the source's read and write. The placement of `cleanup` is semantic. In `refSlice` the mint comes *after* the `cleanup`: a mutable retag through the loaded tag would pop a route tag still on the stack, and a shared one would bury it, so the route's die would no longer find its item on top. Every bracket the compiler opens therefore closes with its route tag on top.

Table 13. Lowering of the remaining statements, extending **STMT-ASSIGN** and **STMT-HALT** of Table 6.

Statement	Emitted sequence
<code>dealloc(p)</code>	Read p into r_v ; $\text{dealloc}(r_v)$.
<code>assignIf(p = w, d := e)</code>	$\text{root}(d)$, before the guard; read p into r_g ; reserve the label j_g ; compile $d := e$ by STMT-ASSIGN patch $K(j_g) = \text{skipIf}(r_g, w, n)$ with n the number of labels the assignment emitted.
<code>pushProtectors</code>	pushProt
<code>popProtectors</code>	popProt

For `assignIf`, the root of the destination is allocated before the guard because a root allocated inside the guarded fragment would be recorded in L at compile time but would exist at run time only when the guard is taken. The guard *reserves* its label and is patched once the body has been compiled, so the body is compiled exactly once and its measured length is the skip count; no statement is compiled twice.

A.5 What remains outside the theorem

Every statement and expression above is inside Theorem 2.9. What the theorem assumes, and where the model approximates Rust, is:

- *Layouts*. The layout environment must be well formed (Definition 2.6). The proof holds it as a hypothesis; for the layouts `rustc` gives, it is discharged by the decidable shape check, program by program.
- *Fat pointers*. A slice reference is one value whose extent travels in its provenance, not a pointer and a length in two words; the extent is recovered whenever the pointer's bytes are read back whole.
- *Wildcard provenance*. An access through an exposed address resolves to one granting item, deterministically (Appendix A.2).
- *Executions*. The theorems preserve successful finite runs; they say nothing of divergence and do not relate source and target failure messages.